

Spicing up Genetic Netlist Generation with LLMs

Stefan Uhlich¹, Yağız Genç^{1,2}, Andrea Bonetti¹,
Arun Venkitaraman¹, Chia-Yu Hsieh¹, Eisaku Ohbuchi³, Lorenzo Servadei^{1,4}
¹Sony AI, Switzerland ²EPFL, Switzerland ³Sony Semiconductor Solutions, Japan ⁴TU Munich, Germany

Abstract

Analog circuit topology synthesis remains challenging because useful designs occupy a tiny fraction of a combinatorial search space, and small structural changes can induce highly nonlinear changes in behavior. Evolutionary algorithms are attractive because they can optimize over discrete circuit topologies using only black-box evaluations, but they often require many SPICE simulations and may converge prematurely. We introduce LLM-SPICEMixer, a hybrid synthesis framework that augments genetic netlist generation with IGEL (Inspiration-Guided Evolution with LLMs), an LLM-based proposal operator. During search, IGEL prompts an LLM with high-performing circuits from the elite set and instructs it to generate a new SPICE netlist, which is then evaluated by SPICE and selected using the same reward mechanism as conventional genetic operators. Thus, the LLM contributes structured topology proposals while simulation remains the source of truth. We evaluate LLM-SPICEMixer on a challenging benchmark task: synthesizing transistor-level circuits that implement a discriminant function for Iris classification. Compared with the genetic framework without LLM guidance, LLM-SPICEMixer improves the median final training reward by 8.4% and the median validation-selected test reward by 8.8%. The best validation-selected circuit achieves 93.3% test accuracy at the nominal tt corner and 85.9% average test accuracy across 17 process, voltage, and temperature corners.

CCS Concepts

• **Hardware** → **Electronic design automation**.

Keywords

Analog circuit synthesis, Genetic algorithms, Large language models, SPICE netlists, Analog discriminant-function synthesis

ACM Reference Format:

Stefan Uhlich, Yağız Genç, Andrea Bonetti, Arun Venkitaraman, Chia-Yu Hsieh, Eisaku Ohbuchi, and Lorenzo Servadei. 2026. Spicing up Genetic Netlist Generation with LLMs. In *2026 ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD '26)*, September 07–09, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3831599.3840324>

1 Introduction

Analog circuit design remains challenging because small structural modifications can lead to highly nonlinear and non-smooth changes in behavior [1]. Circuit synthesis requires both topology selection,

which determines devices and interconnections, and sizing, which sets parameters such as transistor widths and lengths. For fixed topologies, substantial progress has been made using Bayesian optimization [2, 3], reinforcement learning [4–7], and large language models (LLMs) [8–10]. Topology synthesis remains harder because it involves discrete structural choices and is strongly coupled with sizing. Consequently, many successful approaches focus on well-studied circuit families such as operational amplifiers [11–15], filters [16–18], power converters [19–21], or other common circuit classes [22–25]. This leaves open how to synthesize useful circuits for less standardized tasks, where no canonical topology or family-specific search space is available.

Evolutionary algorithms are attractive for open-ended topology synthesis because they optimize candidate circuits directly through SPICE simulation. However, purely genetic search can require many evaluations and may converge prematurely. LLMs offer a complementary capability: they can generate structured text and may capture useful regularities from circuit descriptions and netlists. Yet asking an LLM to design a complete circuit from scratch is unreliable, especially for non-standard tasks where memorized templates are unlikely to apply. This suggests a hybrid strategy: use the LLM not as a standalone circuit designer, but as a proposal mechanism inside a simulation-driven evolutionary loop.

Motivated by this idea, we introduce LLM-SPICEMixer, a circuit synthesis method that extends SPICEMixer [26] with IGEL, an LLM-based proposal operator for *Inspiration-Guided Evolution with LLMs*. During search, IGEL samples high-quality circuits from the elite set and provides them to the LLM as inspirations. The LLM then proposes a new candidate netlist, which is evaluated by SPICE and selected using the same reward mechanism as conventional genetic operators such as mutation, crossover, and pruning. Thus, the LLM injects structured topology variations, while SPICE simulation provides the performance evaluation. The goal is not to replace genetic search, but to make it less prone to premature convergence by adding informed and diverse proposals.

We evaluate LLM-SPICEMixer on an analog discriminant-function synthesis task based on Iris classification. The goal is to synthesize a transistor-level circuit whose input voltages encode the four Iris features and whose output voltages represent the three class scores. This task is useful as a benchmark because it is specified by data-dependent classification behavior rather than standard analog design targets, and suitable topologies are not known a priori. Thus, the method must discover compact transistor-level structures rather than tune a known circuit template.

In summary, the contributions of this paper are as follows:

- We introduce LLM-SPICEMixer, a hybrid synthesis method that augments genetic netlist generation with IGEL, an LLM-based proposal operator that generates candidate circuits from elite netlists within a SPICE-driven evolutionary loop.



This work is licensed under a Creative Commons Attribution 4.0 International License. *MLCAD '26, Jeju Island, Republic of Korea*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2878-5/2026/09
<https://doi.org/10.1145/3831599.3840324>

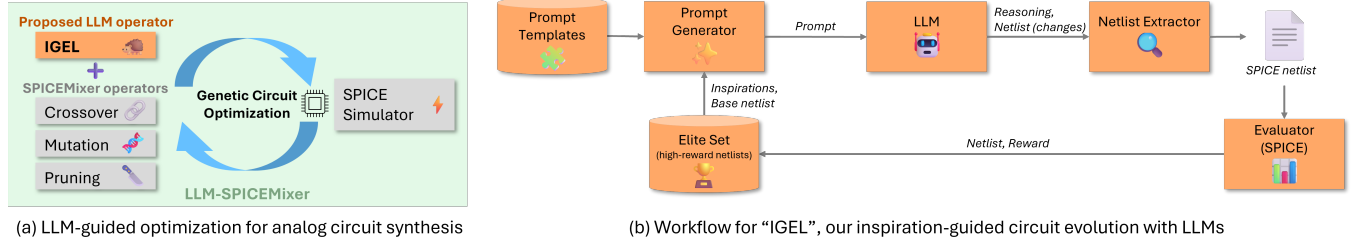


Figure 1: Overview of LLM-SPICEMixer: (a) LLM-augmented genetic circuit optimization loop. (b) IGEL prompts the LLM with inspiration circuits, then evaluates the proposed netlist with SPICE; it is added to the elite set if good enough.

- We propose an analog discriminant-function synthesis benchmark based on the Iris dataset and use it to evaluate open-ended transistor-level topology synthesis.
- We present an empirical study of prompting strategies, decoding settings, model choices, and operator mixtures, showing that IGEL improves search performance when used as a complement to conventional genetic operators.

The paper is organized as follows. Sec. 2 reviews related work. Sec. 3 introduces LLM-SPICEMixer and IGEL. Sec. 4 describes the analog discriminant-function synthesis task and evaluation setup. Sec. 5 presents results, including baseline comparisons and ablations. Finally, Sec. 6 concludes and outlines future work.

2 Related Work

Analog design automation has seen strong progress for sizing fixed topologies using Bayesian optimization, reinforcement learning, and, more recently, LLM-based optimization [2–10]. Topology synthesis is more challenging: early work relied on evolutionary search and genetic programming for joint topology-and-sizing optimization [16–18], while recent methods often focus on specific circuit families, particularly operational amplifiers and power converters [11–15, 19, 21]. Other approaches address more general circuit generation using graph-based, generative, or LLM-based methods [20, 22–25, 27]. Agentic LLM frameworks such as AnalogAgent [28] have also introduced multi-agent workflows for iterative analog circuit generation. These methods typically use learned models or LLMs as primary generators or design agents, whereas our approach uses the LLM only as one proposal operator inside a genetic search.

A separate line of work integrates LLMs into evolutionary search. FunSearch and related systems use an LLM to propose candidates that are evaluated and selected within an iterative search loop [29–35]. Our method follows this paradigm, but transfers it to transistor-level analog synthesis: it builds on SPICEMixer [26], which evolves SPICE netlists directly, and extends it with IGEL, an LLM-based proposal operator that generates candidates from elite netlists. In contrast to general program-search systems, our candidates are physical circuits whose quality must be determined by SPICE simulation under task-specific testbenches and process corners. Related uses of LLM-generated SPICE netlists also appear in SpiceFuzz [36], but there the goal is simulator fuzzing rather than reward-driven functional synthesis. To the best of our knowledge, LLM-SPICEMixer is the first method to embed LLM-generated netlist proposals into an evolutionary loop for analog circuit synthesis.

3 LLM-SPICEMixer

We first briefly review SPICEMixer [26]. We then introduce IGEL, our LLM-based proposal operator, which enriches the elite set during genetic search. The overall system is illustrated in Fig. 1.

3.1 Recap of SPICEMixer

SPICEMixer is a circuit synthesis method based on a genetic algorithm that evolves SPICE netlists directly. Rather than relying on an abstract graph representation or a hand-crafted chromosome encoding, it treats the netlist itself as the genome. This makes the method naturally compatible with arbitrary components and sub-circuits, and therefore easy to adapt to different circuit libraries and process design kits.

SPICEMixer repeatedly applies one of three proposal operators to generate new candidate circuits. The first operator is *crossover*, which combines two elite netlists line by line. The second operator is *mutation*, which combines one elite netlist with a randomly sampled netlist to introduce new structural variations while preserving useful parts of a strong parent. The third operator is *pruning*, which merges compatible component definitions so that the resulting circuit can become smaller and more compact.

Each newly generated netlist is then evaluated by SPICE simulation under a task-specific testbench, and its performance is mapped to a scalar reward that serves as the fitness value. SPICEMixer maintains an elite set containing the best circuits found so far, and parent circuits are sampled from this set using a rank-based roulette-wheel strategy. This biases the search toward high-quality solutions while still preserving diversity.

3.2 Inspiration-Guided Evolution with LLMs

Because circuits can be represented as netlists, LLMs are a natural choice for circuit synthesis. However, their effectiveness depends strongly on how they are used. One option is to ask the LLM to generate a complete circuit directly, as in AnalogCoder [23] and AnalogCoder-Pro [24]. In our setting, however, this strategy is often ineffective. Our goal is to discover novel circuit topologies from uncommon circuit families, and in such cases the model often reproduces the same canonical solution [37, 38], even across multiple interaction rounds and even when given feedback on circuit performance as we will show in Sec. 5.6. Although some methods can mitigate this behavior [39], it remains a general limitation. Another possibility would be to use the LLM as a verification tool, either as a pre-check before SPICE simulation or as a surrogate model for estimating circuit quality. We expect this to be unreliable as well,

because the model would need to assess novel circuits for novel tasks solely from their netlists.

We therefore use the LLM as a proposal operator within a genetic search process, which we call IGEL. In this sense, the LLM is used in a manner conceptually similar to coding-agent systems such as AlphaEvolve [34]. Since LLMs are strong at code generation, we expect them to be well suited for proposing new candidate netlists, especially when diverse prompt formulations are used. To reduce generation collapse, we do not rely on IGEL alone. Instead, we combine it with the original SPICEMixer operators—crossover, mutation, and pruning—as shown in Fig. 1a. In this way, the different operators can complement one another, since each of them can further refine a solution that was produced by another operator in an earlier step.

General Approach. Fig. 1b illustrates our use of the LLM. From the current elite set, which contains the best solutions found so far, we sample $K = 3$ netlists using rank-based roulette-wheel sampling. We populate a prompt template with these netlists and their reward scores, and pass the resulting prompt to the LLM. The model then analyzes the given circuits and, when requested by the prompt, first produces a short plan describing how they should be modified. It subsequently outputs a new candidate netlist. We extract this netlist from the LLM response and evaluate it in a SPICE testbench.

Netlist Preprocessing. We apply two preprocessing steps to improve the effectiveness of the method. First, we identify and remove circuits that share the same topology and differ only in parameter values. By ensuring that the LLM only sees structurally distinct inspirations, we encourage it to propose topological modifications rather than merely adjust transistor sizes. In preliminary experiments, we observed that without this step, the genetic algorithm often converged prematurely to an elite set containing only a single topology with minor sizing variations, many of which were produced by IGEL. Once this occurred, the search rarely recovered from the resulting collapse and often failed to discover better circuits.

Second, rather than providing the original netlists directly, we convert PDK-specific transistor instances, in our case from the SkyWater PDK [40], into a simplified MOS representation. In this process, we remove the explicit bulk connection and rename nets to more descriptive names, for example `net_supply_0` to `net_vdd` and `0` to `net_gnd`. For example, the line

```
X6 net_internal_1 net_internal_0 net_supply_0 net_supply_0
+ sky130_fd_pr__pfet_01v8 w=20 l=1
```

is rewritten as

```
M6 net_internal_1 net_internal_0 net_vdd PMOS w=20 l=1
```

This preprocessing reduces potential confusion caused by uncommon component and net names that may have been underrepresented during LLM training. It also removes ambiguities, for example when `0` could refer either to ground or to a parameter value.

Prompt Templates. To analyze the effect of prompting on synthesis performance, we compare prompt templates that vary along two dimensions:

- “raw” vs. “diff”. In the “raw” setting, the LLM generates a complete candidate netlist. In the “diff” setting, the LLM specifies only the lines that should be changed relative to a given base netlist.
- “with reasoning” vs. “without reasoning”. In the “with reasoning” setting, the prompt asks the LLM to first analyze the given inspirations and describe a modification strategy before generating the output. In the “without reasoning” setting, the LLM is instructed to produce the output directly, without an explicit planning step. These two design choices result in four prompt templates. In Appx. A, Fig. 4 and Fig. 5 show example prompts together with the corresponding LLM outputs.

We expect both explicit reasoning and edit-based generation to improve performance. Asking the LLM to analyze the inspiration circuits before proposing a new netlist may help it identify useful structural patterns. Likewise, the “diff” format is well suited to genetic search because it encourages smaller, more local modifications of high-quality circuits. Based on this intuition, prompt templates with reasoning and structured edits appear to be strong candidates, although the best design remains an empirical question. As shown in Sec. 5.3, explicit reasoning is particularly beneficial, whereas the difference between “raw” and “diff” is comparatively small. Overall, the best performance is obtained by an alternating scheme that cycles between the reasoning-based “raw” and “diff” formats, suggesting that the additional diversity introduced by using two prompt styles is advantageous.

4 Analog Discriminant-Function Synthesis

We evaluate LLM-SPICEMixer on the synthesis of an analog circuit that implements a discriminant function for the Iris classification task [41]. This is an interesting non-standard synthesis problem, because the objective is not to reproduce a known circuit class, but to realize a classifier directly as an analog circuit. Hence, IGEL cannot rely on memorized solutions from the LLM, but must instead infer useful patterns from the inspirations.

4.1 Task Setup

To implement the discriminant function for the Iris dataset, the circuit has four input nets corresponding to the four Iris features (*sepal length (cm)*, *sepal width (cm)*, *petal length (cm)*, and *petal width (cm)*), and three output nets corresponding to the three classes (*setosa*, *versicolor*, and *virginica*). All four features are normalized by min-max scaling computed on the training set. During circuit evaluation, the normalized feature values are mapped to input voltages in the range $[0\text{ V}, 1.8\text{ V}]$. For a given input sample, the predicted class is determined by the output node with the highest voltage.

We use the standard Iris dataset, which contains 150 samples. The data are split into training, validation, and test sets using a stratified 60/20/20 split, yielding 90 training samples, 30 validation samples, and 30 test samples. This split is used such that the search optimizes the training reward, the validation set is used to select the best circuits found during the search, and the test set is reserved exclusively for final evaluation. To evaluate a candidate circuit efficiently, we present all samples from the training and validation splits within a single transient simulation. Each feature is applied through a piecewise-linear voltage source. Each sample is

presented for 8 ns, and the circuit outputs are read at the end of this time window. To reduce artifacts caused by a fixed sample order, we shuffle each split three times with different random seeds and concatenate the resulting sequences. This produces longer input streams while preserving the class distribution. The same shuffling-and-concatenation procedure is applied to the held-out test split during final evaluation.

The synthesized circuits are built entirely from SkyWater transistors [40]. In addition to the topology, that is, the transistor interconnection pattern, the search space also includes transistor width and length as continuous parameters for each device. We attach a small output load capacitance of 10 fF to each output node to model the input capacitance of a subsequent stage.

4.2 Reward and Multi-Corner Evaluation

Each candidate netlist is evaluated with Ngspice [42] under multiple process, voltage, and temperature conditions. We consider one nominal corner, tt at 1.8 V and 25°C, and 16 extreme corners formed by all combinations of process corners ff, ss, sf, and fs, supply voltages 1.62 V and 1.98 V, and temperatures 0°C and 85°C. In total, each circuit is evaluated on 17 corners, and the results are averaged. This encourages the search to find solutions that are robust across operating conditions and reduces the risk that a circuit performs well only because of artifacts of the idealized SPICE compact models.

For each sample, the ideal output is a one-hot voltage vector: the correct class should be close to 1.8 V, while the two incorrect classes should be close to 0 V. Let $y_i \in \mathbb{R}^3$ denote the simulated output voltages for sample i , and let $t_i \in \{0 \text{ V}, 1.8 \text{ V}\}^3$ denote the corresponding one-hot target. For one data split and one corner, we define the reward as $R = \frac{1}{2}(A + M)$, where A is the classification accuracy,

$$A = \frac{1}{N} \sum_{i=1}^N \mathbb{1} \left[\arg \max_j y_{i,j} = \arg \max_j t_{i,j} \right], \quad (1)$$

and M is the target-voltage score,

$$M = 1 - \frac{1}{3N} \sum_{i=1}^N \|y_i - t_i\|_1. \quad (2)$$

Here, N is the number of samples in the split, including the repeated shuffled sequences described above, and $\|\cdot\|_1$ denotes the 1-norm. The maximum possible reward is therefore $R = 1$, while in practice R is smaller. The first term, A in (1), rewards correct predictions. The second term, M in (2), rewards large voltage separation by encouraging the correct output to move toward the supply voltage and the incorrect outputs toward ground. This is important because two circuits can achieve the same accuracy while exhibiting very different output confidence. To obtain a robust classifier, we therefore seek a large output separation score M .

Furthermore, we apply two additional reward penalties to guide the search toward compact circuits that are not only functional but also physically plausible:

- **Validity penalty.** A penalty of 0.05 is applied for each violated structural validity check and may therefore be incurred multiple times by the same netlist. These checks require that all three output nets are present, transistor bulk terminals are connected

correctly, input nets are connected only to transistor gates, no floating nets exist, output nets are connected only to transistor drain or source terminals, and supply or ground nets are not connected to transistor gates.

- **Size penalty.** A penalty of $0.0025 \times N_{\text{transistors}}$ is subtracted to favor smaller circuits, where $N_{\text{transistors}}$ denotes the number of transistors in the netlist, that is, the number of netlist lines.

All reward values reported in this paper are penalized rewards, including both the validity and size penalties described above.

For every candidate, we compute training and validation rewards for each corner and then average them over all 17 corners. The genetic search uses the averaged training reward as the fitness value, while the averaged validation metric is recorded and later used to select the circuit with the highest validation reward. This follows standard machine learning practice and helps reduce overfitting to the training split. If a simulation fails or the output traces cannot be parsed correctly, the candidate receives a reward of -1 , which discourages invalid circuits.

5 Results

In the experiments below, *IGEL* denotes the concrete instantiation of our LLM-based proposal operator within LLM-SPICEMixer. At each IGEL step, we sample $K = 3$ high-quality circuits from the current elite set using rank-based roulette-wheel sampling, pre-process them into a simplified MOS representation, and provide them to the LLM as *inspirations*. The model then proposes one new candidate netlist, which is converted back to the SkyWater PDK representation, evaluated by SPICE, and inserted into the evolutionary search. Unless stated otherwise in the ablation studies, we use the following IGEL configuration: reasoning-enabled prompts with an alternating “raw”/“diff” format (cf., Sec. 5.3), balanced decoding (cf., Sec. 5.4), and Qwen3.5 27B as the proposal model (cf., Sec. 5.5). Each synthesis run is executed for 131,072 proposal steps, where each step generates and evaluates one candidate circuit. The four proposal operators are sampled with relative weights 1 : 1 : 1 : 0.5 for crossover, mutation, pruning, and IGEL, respectively. Thus, IGEL is invoked only half as often as each conventional operator, reducing the computational cost of LLM inference.

For all experiments, we use vLLM [43] to serve the LLMs on GPU servers. Models with up to 12B parameters (Gemma3 270M, 1B, 4B, and 12B, as well as Qwen3.5 9B) are run on Ada6000 GPUs, whereas larger models (Gemma3 27B and Qwen3.5 27B) are run on H200 GPUs. To reduce statistical noise, each method or configuration is evaluated over nine independent runs.

We first compare LLM-SPICEMixer with the baseline that does not use IGEL, namely SPICEMixer, and show that IGEL substantially improves performance. We then compare the best synthesized circuits with standard machine learning baselines, specifically logistic regression and a single-hidden-layer MLP. Finally, we perform ablation studies to isolate the effect of the main design choices in LLM-SPICEMixer, including the prompt template, decoding settings, and model family and size.

5.1 Comparison with Evolutionary Baselines

We first compare LLM-SPICEMixer against two evolutionary baselines. The first is GraCo ES [27], which uses a graph neural network

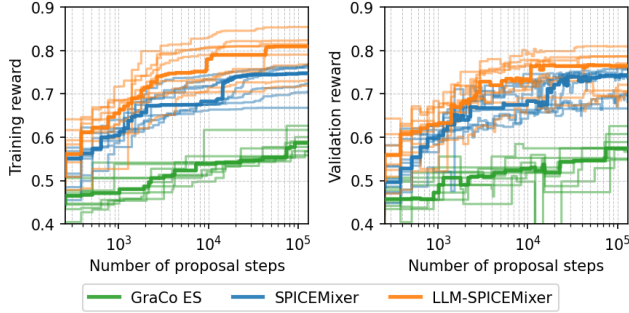


Figure 2: Training and validation rewards of the best-so-far circuit in each run. Circuits are selected by training reward and then evaluated on validation. Each run uses 131,072 proposal steps; thicker lines indicate medians over nine runs.

(GNN) to sequentially construct a graph representation of the synthesized circuit, while an evolutionary strategy (ES) [44] is used to optimize the GNN parameters. The second baseline is SPICEMixer [26], which was reviewed in Sec. 3.1.

The evolution of the reward on the training and validation splits is shown in Fig. 2, and the final training rewards are summarized in Tab. 1. GraCo ES performs worst: the search quickly stalls, and we observe that it produces circuits with highly similar topologies, leading to generation collapse and a median reward of only 0.587. SPICEMixer performs substantially better, achieving a median reward of 0.747. LLM-SPICEMixer yields the strongest overall performance. It improves the median reward to 0.810 (+0.063 over SPICEMixer) and also discovers the best overall circuit, with a reward of 0.855 (+0.087 over the best circuit found by SPICEMixer).

To assess whether this improvement is statistically significant, we perform a one-sided unpaired permutation test on the results of the nine independent runs. The test shows that the median improvement of 0.063 is statistically significant, with a p -value of 0.0052. These results indicate that adding the LLM-based proposal operator substantially improves the search and leads to better final solutions.

Fig. 2 also shows the evolution of the reward curves on the training and validation splits. Each plot reports the reward of the best circuit found so far according to the training set and evaluates that same circuit on the validation set. We observe that LLM-SPICEMixer consistently outperforms SPICEMixer, demonstrating that IGEL benefits the synthesis process. We also observe a train–validation gap: the final median reward reaches 0.81 on the training set, compared to approximately 0.77 on the validation set. Nevertheless, the overall ranking remains unchanged, and LLM-SPICEMixer also produces the best circuits on the validation split.

Using the validation reward, we select the best circuit from each run and evaluate it on the test split. Tab. 2 summarizes these results. Again, LLM-SPICEMixer performs best, with a median test reward of 0.807, which is +0.065 higher than SPICEMixer.

Finally, Fig. 3 shows the schematic of the best circuit on the validation set found by LLM-SPICEMixer. The complete netlist, including transistor sizes, is shown in Fig. 6 in Appx. B. Because of the size penalty in our reward, the resulting circuit is compact while still performing well on the classification task. Interestingly, the best solution in Fig. 3 uses only two of the four available inputs,

Table 1: Final best training reward for different methods. Values are computed over nine independent runs.

	GraCo ES [27]	SPICEMixer [26]	LLM-SPICEMixer
Average $\pm$ Std. Dev.	0.588 $\pm$ 0.022	0.737 $\pm$ 0.031	0.799 $\pm$ 0.039
Minimum	0.558	0.668	0.719
Median	0.587	0.747	0.810
Maximum	0.626	0.768	0.855

Table 2: Test rewards obtained by selecting the checkpoints with the highest validation reward. Values are computed over nine independent runs.

	GraCo ES [27]	SPICEMixer [26]	LLM-SPICEMixer
Average $\pm$ Std. Dev.	0.627 $\pm$ 0.025	0.730 $\pm$ 0.031	0.782 $\pm$ 0.041
Minimum	0.580	0.667	0.715
Median	0.622	0.742	0.807
Maximum	0.670	0.773	0.824

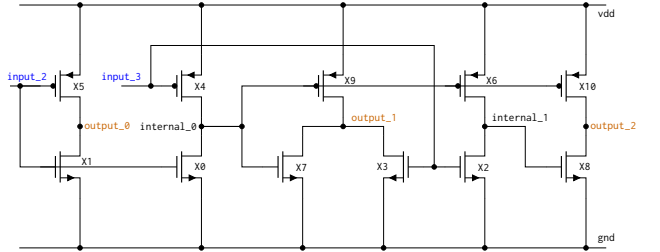


Figure 3: Schematic of the best validation circuit found by LLM-SPICEMixer. The circuit uses only input_2 and input_3, corresponding to petal length and petal width, respectively. The output nets output_0, output_1, and output_2 correspond to setosa, versicolor, and virginica.

namely *petal length* and *petal width*. This is consistent with prior feature-contribution analyses on the Iris dataset, which identify these two attributes as the dominant contributors to the classification decision [45]. It is notable that the synthesized circuit discovers this input subset implicitly and achieves a train accuracy of 93.4%, a validation accuracy of 88.4%, and a test accuracy of 85.9%, as discussed in more detail in the next section. To better understand the circuit behavior, Fig. 8 in Appx. C shows the output waveforms for the tt corner, with the transients for the three shuffled versions of the test split overlaid. The circuit behaves relatively “statically,” which is advantageous because its response does not depend strongly on the cycle at which the samples are presented. In addition, the outputs are clearly separated, indicating a large output separation and thus robustness to noise. For completeness, Fig. 9 in Appx. C shows the corresponding waveforms overlaid across all corners and all shuffles.

Fig. 7 in Appx. B presents three additional strong circuits found by LLM-SPICEMixer. Comparing them shows that our method discovers diverse solutions with substantially different topologies while maintaining good performance. Since this is a non-standard synthesis task for which memorized circuit templates are unlikely to exist, the strong performance of LLM-SPICEMixer suggests that the approach can generalize to novel tasks, which is important for practical analog design.

5.2 Accuracy and Robustness

To place the performance of the synthesized circuits into context, we compare them with two standard machine learning baselines implemented in `scikit-learn` [46]:

- a regularized logistic regression classifier, implemented with `LogisticRegression`, and
- a single-hidden-layer neural network with four hidden units and a batch size of eight implemented with `MLPClassifier`.

The best models are selected on the validation set and then evaluated once on the held-out test set.

For evaluation, we study robustness to perturbations of the input voltages. Specifically, we add Gaussian noise $\mathcal{N}(0, \sigma_{\text{noise}}^2 \mathbf{I})$ to each input sample $\mathbf{x}_i \in \mathbb{R}^4$ with $\sigma_{\text{noise}} \in \{0 \text{ V}, 0.1 \text{ V}, 0.2 \text{ V}, 0.3 \text{ V}, 0.4 \text{ V}, 0.5 \text{ V}\}$. For each noise level, we report results averaged over 16 independent noise realizations. This setup models practical input distortions, such as sensor noise, which can lead to slightly perturbed voltage levels. A practically useful circuit should therefore be robust to such variations.

The results are shown as boxplots in Fig. 10 in Appx. D for both the nominal `tt` corner and the multi-corner setting. In addition, Tab. 3 in Appx. D reports the corresponding test accuracies. Overall, the synthesized circuits show a degradation trend comparable to the ML baselines. In the nominal `tt` setting, the two best circuits are competitive with the baselines and achieve higher median accuracy at several larger noise levels. We hypothesize that this is related to optimizing the reward across multiple corners during synthesis.

5.3 Ablation: Prompting Strategy

We explored the prompt templates introduced in Sec. 3.2 to understand how the LLM query affects synthesis. In addition to fixed templates, we evaluated an alternating scheme that switches between the “raw”-style and “diff”-style prompts.

The results over nine runs using Gemma3 12B are shown in Tab. 4 in Appx. E. Overall, prompt design measurably affects the final reward. Variants with explicit reasoning tend to achieve higher median reward than those without reasoning. Comparing the two output formats, the “diff”-style prompt is slightly more robust than the “raw”-style prompt, especially without reasoning. This is consistent with edit-based outputs encouraging local modifications of elite circuits, which is advantageous for genetic algorithms. Among the evaluated settings, the alternating scheme with reasoning achieves the highest median reward. These results suggest that IGEL is sensitive to how the generation task is framed, and that better prompt design may further improve search performance.

5.4 Ablation: Decoding Settings

We also explored several LLM decoding settings instead of fixing a single choice a priori. In particular, we varied the softmax temperature T and the top- p probability p_{top} [47] to cover a range from nearly deterministic decoding to more diverse sampling. These settings control the trade-off between output stability and diversity and may therefore affect proposal quality.

The results over nine runs using Gemma3 12B are shown in Tab. 5 in Appx. E. We observe some variation across settings, but less than for the prompt templates. The balanced setting with $T = 0.7$ and $p_{\text{top}} = 0.9$ yields the highest median reward, while the other settings

remain in a similar range. Overall, IGEL does not require highly specialized decoding settings. Moderate stochasticity appears to be a reasonable default for this task.

5.5 Ablation: Model Choice and Size

Finally, we explored model family and size using Gemma3 [48] and Qwen3.5 [49]. The results are reported in Tab. 6 in Appx. E.

Overall, performance depends on both model family and size. Within Gemma3, larger models generally perform better, although not strictly monotonically for every statistic. Across all models, Qwen3.5 27B performs best, reaching the highest median and maximum reward. Compared with Gemma3 27B, it improves the median reward by +0.05. We attribute this to overall model quality, reflecting the advantages of a newer model with stronger benchmark performance. We therefore use Qwen3.5 27B as the IGEL model in the main comparison.

To better understand model differences, we also analyzed response lengths in Tab. 8 in Appx. F. Gemma3 produces relatively short outputs, whereas Qwen generates substantially longer responses, partly due to explicit thinking tokens. In particular, Qwen3.5 27B generates much longer responses than the other models. This may indicate that additional test-time reasoning is useful, although our results show only correlation, not causation.

5.6 Ablation: Operator Mixture

We further ablate whether IGEL should replace the conventional SPICEMixer operators or complement them. We compare the default operator mixture against an IGEL-only variant without crossover, mutation, or pruning. The IGEL-only variant runs for 18,816 proposal steps, matching the LLM-call budget of one full LLM-SPICE-Mixer run. Both settings use the same reasoning-enabled alternating “raw”/“diff” prompts with $K = 3$ inspirations.

The results are reported in Tab. 7 in Appx. E. IGEL alone performs substantially worse than the operator mixture. At the same proposal budget of 18,816 steps, the median reward decreases from 0.689 to 0.619. The gap is even larger relative to the longer 131,072-step operator-mixture run with the same LLM-call budget, whose median reward is 0.763. Notably, the best IGEL-only run (0.656) remains below the worst operator-mixture run at the same proposal budget (0.665). These results show that IGEL is most effective as a complementary proposal operator: it injects useful structured variations, while conventional genetic operators remain important for local refinement, recombination, and search dynamics.

6 Conclusions and Outlook

We introduced LLM-SPICEMixer, a hybrid analog circuit synthesis method that augments SPICEMixer with IGEL, an LLM-based proposal operator. Rather than asking the LLM to generate circuits from scratch, we use it to propose new candidate netlists from elite solutions within a genetic search loop. This enables the LLM to contribute structured variations, while SPICE simulation remains the source of truth for evaluating circuit quality.

We evaluated the method on analog discriminant-function synthesis for the Iris classification task. The results show that IGEL improves search performance over SPICEMixer, reduces premature convergence, and yields compact transistor-level circuits with

strong performance. Ablations show that prompt design and model choice are important in this setting, while the decoding configuration has a comparatively smaller effect.

Several directions remain for future work. First, the gap between training and validation/test reward suggests that improved reward design or explicit regularization could further enhance generalization. Second, robustness could be strengthened by incorporating noisy or perturbed inputs directly during synthesis rather than only during post-training evaluation. Third, it would be interesting to explore richer LLM-based proposal strategies, such as verbalized sampling [39], self-reflective refinement [32, 50], or multi-stage proposal-and-repair schemes [24, 51].

Overall, our results suggest that for non-standard analog synthesis tasks, LLMs are most effective not as standalone designers, but as proposal generators embedded within a simulation-driven evolutionary search loop.

References

- [1] B. Razavi, *Design of Analog CMOS Integrated Circuits*, 2nd ed. McGraw-Hill Education, 2017.
- [2] W. Lyu, P. Xue, F. Yang, C. Yan, Z. Hong, X. Zeng, and D. Zhou, "An efficient bayesian optimization approach for automated optimization of analog circuits," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 65, no. 6, pp. 1954–1967, 2017.
- [3] T. Gu, J. Wang, Z. Bi, C. Yan, F. Yang, Y. Qin, T. Cui, and X. Zeng, "tss-bo: Scalable bayesian optimization for analog circuit sizing via truncated subspace sampling," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2024, pp. 1–6.
- [4] K. Settalur, A. Haj-Ali, Q. Huang, K. Hakhamaneshi, and B. Nikolic, "Autockt: deep reinforcement learning of analog circuit designs," in *Proceedings of the 23rd Conference on Design, Automation and Test in Europe*, 2020, pp. 490–495.
- [5] H. Wang, K. Wang, J. Yang, L. Shen, N. Sun, H.-S. Lee, and S. Han, "Gcn-rl circuit designer: Transferable transistor sizing with graph neural networks and reinforcement learning," in *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2020, pp. 1–6.
- [6] A. F. Budak, P. Bhansali, B. Liu, N. Sun, D. Z. Pan, and C. V. Kashyap, "Dnn-opt: An rl inspired optimization for analog circuit sizing using deep neural networks," in *Proceedings of the 58th Annual ACM/IEEE Design Automation Conference*, 2021, pp. 1219–1224.
- [7] S. Kim, Z. Wang, S. Lee, Y. Oh, H. Zhu, D. Kim, and D. Z. Pan, "Ppaas: Pvt and pareto aware analog sizing via goal-conditioned reinforcement learning," in *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2025, pp. 1–9.
- [8] N. K. Somayaji and P. Li, "Llm-uso: Large language model-based universal sizing optimizer," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2025.
- [9] C. Liu and D. Chitnis, "Eesizer: Llm-based ai agent for sizing of analog and mixed signal circuit," *IEEE Transactions on Circuits and Systems I: Regular Papers*, 2025.
- [10] M. Ahmadzadeh, K. Chen, and G. Gielen, "Anaflo: Agentic llm-based workflow for reasoning-driven explainable and sample-efficient analog circuit sizing," in *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2025, pp. 1–7.
- [11] J. Lu, L. Lei, F. Yang, L. Shang, and X. Zeng, "Topology optimization of operational amplifier in continuous space via graph embedding," in *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2022, pp. 142–147.
- [12] Z. Zhao and L. Zhang, "Analog integrated circuit topology synthesis with deep reinforcement learning," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 12, pp. 5138–5151, 2022.
- [13] Z. Chen, S. Meng, F. Yang, L. Shang, and X. Zeng, "Total: Topology optimization of operational amplifier via reinforcement learning," in *2023 24th International Symposium on Quality Electronic Design (ISQED)*. IEEE, 2023, pp. 1–8.
- [14] S. Poddar, A. Budak, L. Zhao, C.-H. Hsu, S. Maji, K. Zhu, Y. Jia, and D. Z. Pan, "A data-driven analog circuit synthesizer with automatic topology selection and sizing," in *2024 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2024, pp. 1–6.
- [15] J. Shen, F. Yang, L. Shang, Z. Bi, C. Yan, D. Zhou, and X. Zeng, "Into-oa: Interpretable topology optimization for operational amplifiers," in *2025 Design, Automation & Test in Europe Conference (DATE)*. IEEE, 2025, pp. 1–7.
- [16] J. R. Koza, F. H. Bennett III, D. Andre, and M. A. Keane, "Automated design of both the topology and sizing of analog electrical circuits using genetic programming," in *Artificial intelligence in design '96*. Springer, 1996, pp. 151–170.
- [17] J. Hu, X. Zhong, and E. D. Goodman, "Open-ended robust design of analog filters using genetic programming," in *Proceedings of the 7th annual conference on Genetic and evolutionary computation*, 2005, pp. 1619–1626.
- [18] Ž. Rojec, J. Olenšek, and I. Fajfar, "Analog circuit topology representation for automated synthesis and optimization," *Informacije MIDE*, vol. 48, no. 1, pp. 29–40, 2018.
- [19] S. Fan, N. Cao, S. Zhang, J. Li, X. Guo, and X. Zhang, "From specification to topology: Automatic power converter design via reinforcement learning," in *2021 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 2021, pp. 1–9.
- [20] P. Vijayaraghavan, L. Shi, E. Degan, V. Mukherjee, and X. Zhang, "Autocircuit-rl: Reinforcement learning-driven llm for automated circuit topology generation," in *International Conference on Machine Learning*. PMLR, 2025, pp. 61 498–61 512.
- [21] J. Gao, Y. Zou, A. Pradhan, W. Huang, Y. Su, K. Yang, and X. Zhang, "Powergenie: Analytically-guided evolutionary discovery of superior reconfigurable power converters," *arXiv preprint arXiv:2601.21984*, 2026.
- [22] J. Gao, W. Cao, J. Yang, and X. Zhang, "Analoggenie: A generative engine for automatic discovery of analog circuit topologies," *arXiv preprint arXiv:2503.00205*, 2025.
- [23] Y. Lai, S. Lee, G. Chen, S. Poddar, M. Hu, D. Z. Pan, and P. Luo, "Analogcoder: Analog circuit design via training-free code generation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 1, 2025, pp. 379–387.
- [24] Y. Lai, S. Poddar, S. Lee, G. Chen, M. Hu, B. Yu, P. Luo, and D. Z. Pan, "Analogcoder-pro: Unifying analog circuit generation and optimization via multi-modal llms," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2026.
- [25] S. Kim, M. Kim, Y. Lee, and Y. Kim, "Analogtobi: Device-level analog circuit topology generation via bipartite graph and grammar guided decoding," *arXiv preprint arXiv:2603.08720*, 2026.
- [26] S. Uhlich, A. Bonetti, A. Venkitaraman, C.-Y. Hsieh, Y. Gençer, M. E. Gürsoy, R. Matsuo, and L. Servadei, "Spicemixer-netlist-level circuit evolution," *arXiv preprint arXiv:2506.01497*, 2025.
- [27] S. Uhlich, A. Bonetti, A. Venkitaraman, A. Momeni, R. Matsuo, C.-Y. Hsieh, E. Ohbuchi, and L. Servadei, "Graco—a graph composer for integrated circuits," *arXiv preprint arXiv:2411.13890*, 2024.
- [28] Z. Bao, Z. Lin, J. Wang, J. Hu, Y. Gao, Y. Wu, X. Li, and X. Xu, "Analogagent: Self-improving analog circuit design automation with llm agents," *arXiv preprint arXiv:2603.23910*, 2026.
- [29] B. Romera-Paredes, M. Barekatin, A. Novikov, M. Balog, M. P. Kumar, E. Dupont, F. J. R. Ruiz, J. S. Ellenberg, P. Wang, O. Fawzi, P. Kohli, and A. Fawzi, "Mathematical discoveries from program search with large language models," *Nature*, vol. 625, pp. 468–475, 2024.
- [30] F. Liu, X. Tong, M. Yuan, and Q. Zhang, "Algorithm evolution using large language model," 2023.
- [31] F. Liu, X. Tong, M. Yuan, X. Lin, F. Luo, Z. Wang, Z. Lu, and Q. Zhang, "Evolution of heuristics: Towards efficient automatic algorithm design using large language model," 2024.
- [32] H. Ye, J. Wang, Z. Cao, F. Berto, C. Hua, H. Kim, J. Park, and G. Song, "Reevo: Large language models as hyper-heuristics with reflective evolution," 2024.
- [33] N. van Stein and T. Bäck, "Llamae: A large language model evolutionary algorithm for automatically generating metaheuristics," 2024.
- [34] A. Novikov, N. Vu, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. R. Ruiz, A. Mehrabian, M. P. Kumar, A. See, S. Chaudhuri, G. Holland, A. Davies, S. Nowozin, P. Kohli, and M. Balog, "Alphaevolve: A coding agent for scientific and algorithmic discovery," 2025.
- [35] X. Zhang, X. Chen, F. Portet, and M. Peyrard, "What makes an llm a good optimizer? a trajectory analysis of llm-guided evolutionary search," 2026.
- [36] Z. Ren, H. Liu, S. Guo, Y. Guo, X. Li, and H. Jiang, "Spicefuzz: Llm-based fuzzing for spice circuit simulator tools bug detection," *ACM Transactions on Design Automation of Electronic Systems*, 2026.
- [37] D. Wright, S. Masud, J. Moore, S. Yadav, M. Antoniak, P. E. Christensen, C. Y. Park, and I. Augenstein, "Epistemic diversity and knowledge collapse in large language models," *arXiv preprint arXiv:2510.04226*, 2025.
- [38] L. Yun, C. An, Z. Wang, L. Peng, and J. Shang, "The price of format: Diversity collapse in llms," *arXiv preprint arXiv:2505.18949*, 2025.
- [39] J. Zhang, S. Yu, D. Chong, A. Sicilia, M. R. Tomz, C. D. Manning, and W. Shi, "Verbalized sampling: How to mitigate mode collapse and unlock llm diversity," *arXiv preprint arXiv:2510.01171*, 2025.
- [40] Google and SkyWater Technology Foundry, "Skywater 130nm PDK," 2020. [Online]. Available: <https://github.com/google/skywater-pdk>
- [41] R. A. Fisher, "The use of multiple measurements in taxonomic problems," *Annals of eugenics*, vol. 7, no. 2, pp. 179–188, 1936.
- [42] Ngspice Contributors, *Ngspice User's Manual*, version 46 ed., Ngspice Project, 2026, accessed: 2026-05-08. [Online]. Available: <https://ngspice.sourceforge.io/docs/ngspice-manual.pdf>
- [43] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving

- with pagedattention,” in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626.
- [44] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, “Evolution strategies as a scalable alternative to reinforcement learning,” *arXiv preprint arXiv:1703.03864*, 2017.
 - [45] A. Palczewska, J. Palczewski, R. Marchese Robinson, and D. Neagu, “Interpreting random forest classification models using a feature contribution method,” in *Integration of reusable systems*. Springer, 2014, pp. 193–218.
 - [46] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
 - [47] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The curious case of neural text degeneration,” in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. [Online]. Available: <https://openreview.net/forum?id=rygGQyrFvH>
 - [48] Gemma Team, “Gemma 3 technical report,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.19786>
 - [49] Qwen Team, “Qwen3.5: Towards native multimodal agents,” Feb. 2026. [Online]. Available: <https://qwen.ai/blog?id=qwen3.5>
 - [50] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang *et al.*, “Self-refine: Iterative refinement with self-feedback,” *Advances in neural information processing systems*, vol. 36, pp. 46 534–46 594, 2023.
 - [51] H. Zhang, S. Sun, Y. Lin, R. Wang, and J. Bian, “Analogxpert: Automating analog topology synthesis by incorporating circuit design expertise into large language models,” in *2025 International Symposium of Electronics Design Automation (ISED)*. IEEE, 2025, pp. 772–777.

A Examples of LLM Prompts and Outputs

Fig. 4 and Fig. 5 on the following pages show example prompts used for IGEL together with the corresponding LLM outputs. These examples illustrate how the different prompt styles are formulated and what kinds of netlist modifications the model proposes.

B Netlists of the Best Validation Circuits

Fig. 6 shows the transistor-level netlist of the circuit with the highest validation reward. Averaged over all 17 process, voltage, and temperature corners, this circuit achieves train/validation/test rewards of 0.855/0.811/0.780. The per-corner accuracies listed above the netlist provide additional detail on its robustness across operating conditions. In addition, Fig. 7 shows three more circuits with strong validation performance. Comparing them further illustrates the diversity of solutions discovered by LLM-SPICEMixer.

Best validation reward circuit			
* Corner tt	Vdd=1.80V Temp=25.00°C:	TrainAcc: 0.96667, ValidAcc: 0.93333, TestAcc: 0.93333	
* Corner ff	Vdd=1.62V Temp= 0.00°C:	TrainAcc: 0.92222, ValidAcc: 0.90000, TestAcc: 0.77778	
* Corner ff	Vdd=1.62V Temp=85.00°C:	TrainAcc: 0.96667, ValidAcc: 0.93333, TestAcc: 0.90000	
* Corner ff	Vdd=1.98V Temp= 0.00°C:	TrainAcc: 0.96667, ValidAcc: 0.93333, TestAcc: 0.92222	
* Corner ff	Vdd=1.98V Temp=85.00°C:	TrainAcc: 0.92222, ValidAcc: 0.80000, TestAcc: 0.85556	
* Corner ss	Vdd=1.62V Temp= 0.00°C:	TrainAcc: 0.92222, ValidAcc: 0.87778, TestAcc: 0.76667	
* Corner ss	Vdd=1.62V Temp=85.00°C:	TrainAcc: 0.93704, ValidAcc: 0.91111, TestAcc: 0.78889	
* Corner ss	Vdd=1.98V Temp= 0.00°C:	TrainAcc: 0.96667, ValidAcc: 0.93333, TestAcc: 0.92222	
* Corner ss	Vdd=1.98V Temp=85.00°C:	TrainAcc: 0.92222, ValidAcc: 0.80000, TestAcc: 0.85556	
* Corner sf	Vdd=1.62V Temp= 0.00°C:	TrainAcc: 0.86667, ValidAcc: 0.84444, TestAcc: 0.77778	
* Corner sf	Vdd=1.62V Temp=85.00°C:	TrainAcc: 0.86667, ValidAcc: 0.86667, TestAcc: 0.77778	
* Corner sf	Vdd=1.98V Temp= 0.00°C:	TrainAcc: 0.96667, ValidAcc: 0.93333, TestAcc: 0.93333	
* Corner sf	Vdd=1.98V Temp=85.00°C:	TrainAcc: 0.96667, ValidAcc: 0.93333, TestAcc: 0.93333	
* Corner fs	Vdd=1.62V Temp= 0.00°C:	TrainAcc: 0.96667, ValidAcc: 0.93333, TestAcc: 0.86667	
* Corner fs	Vdd=1.62V Temp=85.00°C:	TrainAcc: 0.96667, ValidAcc: 0.93333, TestAcc: 0.93333	
* Corner fs	Vdd=1.98V Temp= 0.00°C:	TrainAcc: 0.91852, ValidAcc: 0.80000, TestAcc: 0.85556	
* Corner fs	Vdd=1.98V Temp=85.00°C:	TrainAcc: 0.87407, ValidAcc: 0.76667, TestAcc: 0.80000	
* Average over all corners:		TrainAcc: 0.93442, ValidAcc: 0.88431, TestAcc: 0.85882	
X0	internal_0 input_2 gnd gnd sky130_fd_pr__nfet_01v8 w=51 l=1		
X1	output_0 input_2 gnd gnd sky130_fd_pr__nfet_01v8 w=23 l=1		
X2	internal_1 input_3 gnd gnd sky130_fd_pr__nfet_01v8 w=10 l=1		
X3	output_1 input_3 gnd gnd sky130_fd_pr__nfet_01v8 w=26 l=1		
X4	internal_0 input_3 vdd vdd sky130_fd_pr__pfet_01v8 w=35 l=1		
X5	output_0 input_2 vdd vdd sky130_fd_pr__pfet_01v8 w=27 l=1		
X6	internal_1 internal_0 vdd vdd sky130_fd_pr__pfet_01v8 w=20 l=1		
X7	output_1 internal_0 gnd gnd sky130_fd_pr__nfet_01v8 w=6 l=1		
X8	output_2 internal_1 gnd gnd sky130_fd_pr__nfet_01v8 w=22 l=1		
X9	output_1 internal_0 vdd vdd sky130_fd_pr__pfet_01v8 w=49 l=1.2		
X10	output_2 internal_0 vdd vdd sky130_fd_pr__pfet_01v8 w=36 l=1.2		

Figure 6: Netlist of the circuit with the best validation reward found by LLM-SPICEMixer.

Prompt (w/o reasoning or w/ reasoning)

You are an expert CMOS analog IC designer and SPICE netlist generator. Your job is to design circuits as SPICE netlists.

Global rules (apply to EVERY reply):

- * You may only instantiate the following MOSFETs:
 - * M<index> <drain> <gate> <source> PMOS w=<width> l=<length>
 - * M<index> <drain> <gate> <source> PMOS w=<width> l=<length>

* You must output EXACTLY ONE markdown fenced code block of the form:

```
```spice
...
```
```

* Outside that fenced code block:

- * Output NOTHING. No text before it, no text after it.
- * No explanations, comments, markdown.

* Before that fenced code block:

- * Output your thoughts: Analyse the inspirations, then come up with a strategy that you can use.

* Inside the 'spice' block:

- * Write ONE transistor instance per line.
- * Each transistor line has THREE connections: <drain> <gate> <source>.
- * Do NOT include comments or natural language.

* The transistor width w and length l are positive, real-valued values that you can optimize.

Reason through the design silently. Do NOT show your reasoning steps; only output the final netlist in a single 'spice' block.

Please design a circuit for the task: Analog classifier implementing a discriminant function for Iris with 4 input nets and 3 output nets.

Input nets:

```
- 'net_input_0'
- 'net_input_1'
- 'net_input_2'
- 'net_input_3'
```

Output nets:

```
- 'net_output_0'
- 'net_output_1'
- 'net_output_2'
```

Supply nets:

```
- 'net_vdd'
- 'net_gnd'
```

The circuit is represented as a netlist and is evaluated using SPICE with a reward in [-1, 1], where higher is better.

Take inspiration from the following netlists and try to improve their structure and performance. Combine or mutate useful patterns to design a better circuit. You can do this by copying, removing, or modifying transistor lines from these inspirations.

Inspiration netlist with reward 0.701:

```
```spice
M0 net_internal_0 net_input_2 net_internal_1 NMOS w=25 l=3
M1 net_internal_2 net_input_3 net_output_2 NMOS w=34.7 l=54.8
M2 net_output_2 net_input_3 net_output_2 NMOS w=21.1 l=54.8
M3 net_output_2 net_input_3 net_output_2 NMOS w=21.1 l=54.8
M4 net_gnd net_internal_2 net_internal_1 NMOS w=34.7 l=68.1
M5 net_gnd net_internal_2 net_internal_1 NMOS w=34.7 l=68.1
M6 net_gnd net_internal_3 net_output_0 NMOS w=21.1 l=12.9
M7 net_internal_0 net_internal_3 net_output_0 NMOS w=16.7 l=12.9
M8 net_internal_0 net_internal_3 net_output_0 NMOS w=16.7 l=12.9
M9 net_internal_0 net_internal_3 net_output_0 NMOS w=16.7 l=12.9
M10 net_internal_0 net_internal_3 net_output_0 NMOS w=16.7 l=12.9
M11 net_output_1 net_internal_2 net_output_1 NMOS w=9.7 l=41.5
```
```

Inspiration netlist with reward 0.703:

```
```spice
M0 net_internal_0 net_input_2 net_internal_1 NMOS w=25 l=3
M1 net_internal_2 net_input_3 net_output_2 NMOS w=34.7 l=54.8
M2 net_output_2 net_input_3 net_output_2 NMOS w=21.1 l=54.8
M3 net_gnd net_internal_2 net_internal_1 NMOS w=34.7 l=68.1
M4 net_internal_0 net_internal_3 net_output_0 NMOS w=16.7 l=12.9
M5 net_internal_0 net_internal_3 net_output_0 NMOS w=16.7 l=12.9
M6 net_internal_1 net_internal_2 net_output_2 NMOS w=34.7 l=68.1
M7 net_output_1 net_internal_2 net_output_1 NMOS w=9.7 l=41.5
```
```

Inspiration netlist with reward 0.706:

```
```spice
M0 net_gnd net_input_3 net_internal_0 PMOS w=95.3 l=36.1
M1 net_internal_0 net_input_2 net_internal_1 NMOS w=25 l=3
M2 net_internal_1 net_input_1 net_internal_2 PMOS w=72.7 l=5.35
M3 net_internal_2 net_input_3 net_output_2 NMOS w=34.7 l=54.8
M4 net_output_2 net_input_3 net_output_2 NMOS w=21.1 l=54.8
M5 net_gnd net_internal_2 net_internal_1 NMOS w=34.7 l=68.1
M6 net_internal_0 net_internal_3 net_output_0 NMOS w=16.7 l=12.9
M7 net_internal_0 net_internal_3 net_output_0 NMOS w=16.7 l=12.9
M8 net_internal_0 net_internal_3 net_output_0 NMOS w=16.7 l=12.9
M9 net_internal_1 net_internal_2 net_output_2 NMOS w=34.7 l=68.1
M10 net_output_1 net_internal_2 net_output_1 NMOS w=9.7 l=41.5
```
```

Now propose a NEW netlist that is likely to achieve a higher reward, while respecting all global rules from the system prompt. The new circuit SHOULD NOT exceed the size of the inspiration netlists and have a MAXIMUM of 13 transistors or LESS. Avoid copying a netlist and only changing transistor widths and lengths but also consider topological changes. A "topology change" means any structural modification to the circuit graph, including at least one of:

- * Rewire: change at least one of 'drain', 'gate', 'source' of any transistor to a different net (not just 'w/l').
- * Add device: insert a new transistor (new line) and connect it to existing nets and/or a new internal net.
- * Remove device: delete an existing transistor (remove a line).
- * Split/Merge nets: introduce a new internal net to break a previously shared node, or connect two previously distinct internal nets into one by rewiring.
- * Change device type: replace an 'NMOS' with a 'PMOS' (or vice versa) and rewire if needed.

Remember: your reply must consist ONLY of a single 'spice' fenced code block containing the netlist and nothing else.

Remember: First reason, then answer with a 'spice' fenced code block containing the netlist and nothing else.

LLM Answer (w/ reasoning) and Netlist (generic SPICE components)

Okay, let's analyze the provided inspiration netlists and devise a strategy for a potentially improved classifier. The common elements across the three are the use of internal nodes and repeated transistor structures for certain outputs, suggesting an attempt at redundancy or amplification. However, the redundancy isn't particularly efficient, and the structures are somewhat repetitive. The reward scores are quite similar, indicating that small changes might be beneficial.

My strategy will be to simplify the circuit while retaining the core concepts of using internal nodes and multiple transistors for each output. I'll focus on reducing redundancy and introducing a different combination of NMOS and PMOS transistors to potentially enhance the discriminant function. Specifically, I'll try to incorporate a PMOS transistor to create a more balanced circuit. I'll also reduce the number of transistors slightly to see if it improves performance.

```
```spice
M0 net_gnd net_input_3 net_internal_0 PMOS w=95.3 l=36.1
M1 net_internal_0 net_input_2 net_internal_1 NMOS w=25 l=3
M2 net_internal_1 net_input_1 net_internal_2 PMOS w=72.7 l=5.35
M3 net_internal_2 net_input_3 net_output_2 NMOS w=34.7 l=54.8
M4 net_internal_0 net_internal_3 net_output_0 NMOS w=16.7 l=12.9
M5 net_internal_0 net_internal_3 net_output_0 NMOS w=16.7 l=12.9
M6 net_internal_1 net_internal_2 net_output_2 NMOS w=34.7 l=68.1
M7 net_output_1 net_internal_2 net_output_1 NMOS w=9.7 l=41.5
```
```

Extracted Netlist (Skywater-PDK): Reward = 0.720

```
X0 0 net_input_3 net_internal_0 net_supply_0
+ sky130_fd_pr__pfet_01v8 w=95.3 l=36.1
X1 net_internal_0 net_input_2 net_internal_1 0
+ sky130_fd_pr__nfet_01v8 w=25 l=3
X2 net_internal_1 net_input_1 net_internal_2 net_supply_0
+ sky130_fd_pr__pfet_01v8 w=72.7 l=5.35
X3 net_internal_2 net_input_3 net_output_2 0
+ sky130_fd_pr__nfet_01v8 w=34.7 l=54.8
X4 net_internal_0 net_internal_3 net_output_0 0
+ sky130_fd_pr__nfet_01v8 w=16.7 l=12.9
X5 net_internal_0 net_internal_3 net_output_0 0
+ sky130_fd_pr__nfet_01v8 w=16.7 l=12.9
X6 net_internal_1 net_internal_2 net_output_2 0
+ sky130_fd_pr__nfet_01v8 w=34.7 l=68.1
X7 net_output_1 net_internal_2 net_output_1 0
+ sky130_fd_pr__nfet_01v8 w=9.7 l=41.5
```

Figure 4: LLM prompt and answer for the "raw"-style template version.

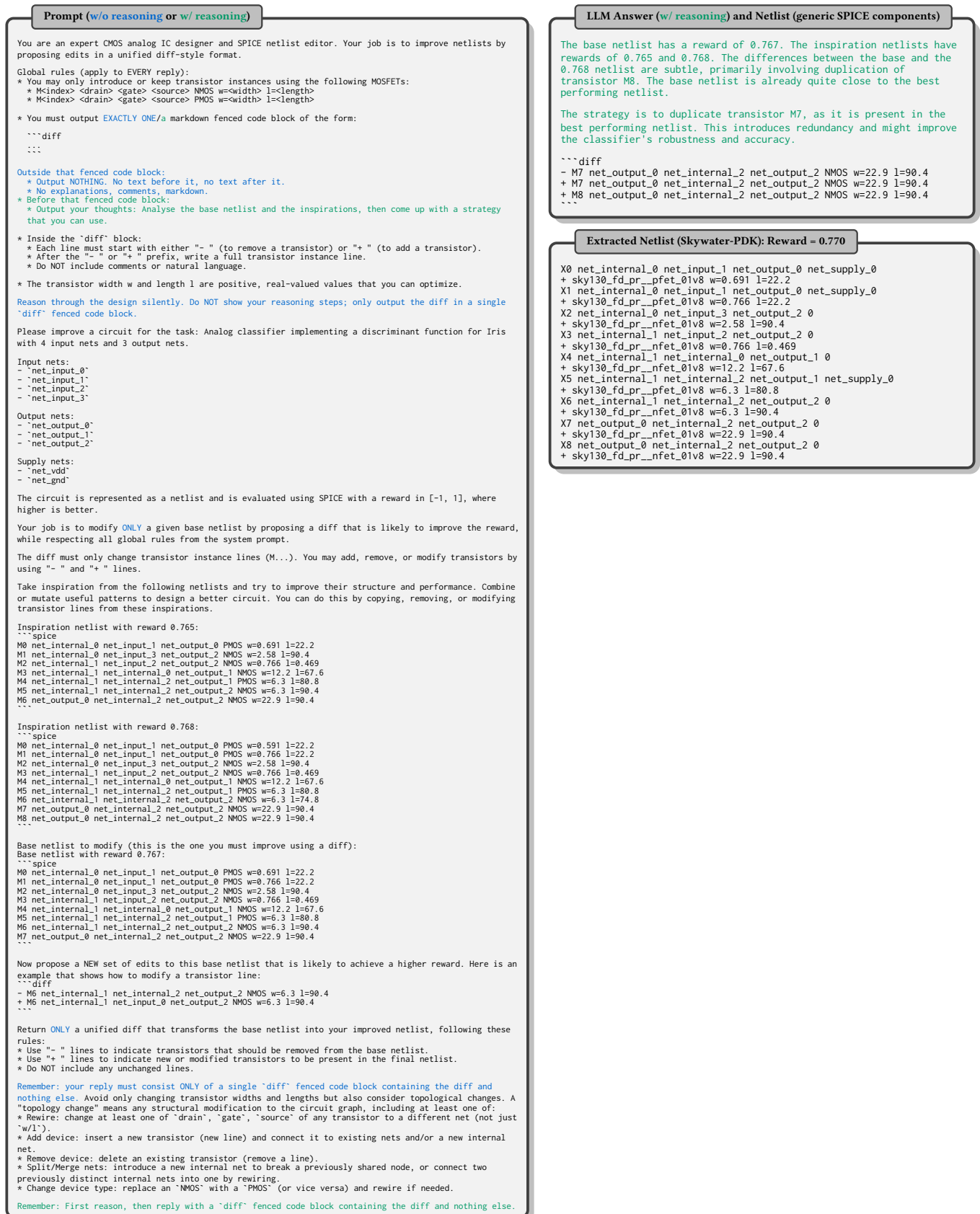


Figure 5: LLM prompt and answer for the “diff”-style template

Second best validation reward circuit with train / validation / test reward of 0.815 / 0.787 / 0.760

```

X0 internal_0 input_2 gnd gnd sky130_fd_pr__nfet_01v8 w=31 l=1
X1 output_0 input_2 gnd gnd sky130_fd_pr__nfet_01v8 w=13 l=1
X2 output_1 input_3 gnd gnd sky130_fd_pr__nfet_01v8 w=3.5 l=1
X3 internal_0 input_2 vdd vdd sky130_fd_pr__pfet_01v8 w=3.5 l=1
X4 internal_1 input_0 vdd gnd sky130_fd_pr__nfet_01v8 w=3.5 l=0.17
X5 output_0 input_2 vdd vdd sky130_fd_pr__pfet_01v8 w=3.5 l=1
X6 output_2 input_3 vdd gnd sky130_fd_pr__nfet_01v8 w=38.2 l=7
X7 output_1 internal_0 gnd gnd sky130_fd_pr__nfet_01v8 w=2 l=1
X8 output_2 internal_1 gnd gnd sky130_fd_pr__nfet_01v8 w=2 l=1
X9 output_0 internal_1 output_2 vdd sky130_fd_pr__pfet_01v8 w=70 l=17
X10 output_1 internal_0 vdd vdd sky130_fd_pr__pfet_01v8 w=5 l=1

```

Third best validation reward circuit with train / validation / test reward of 0.795 / 0.784 / 0.753

```

X0 output_1 input_1 gnd gnd sky130_fd_pr__nfet_01v8 w=10 l=2
X1 output_2 input_1 gnd gnd sky130_fd_pr__nfet_01v8 w=20 l=2
X2 internal_0 input_2 gnd gnd sky130_fd_pr__nfet_01v8 w=25 l=0.211
X3 output_0 input_2 gnd gnd sky130_fd_pr__nfet_01v8 w=20 l=2
X4 output_1 input_3 gnd gnd sky130_fd_pr__nfet_01v8 w=10 l=2
X5 internal_0 input_2 output_0 vdd sky130_fd_pr__pfet_01v8 w=25 l=0.211
X6 output_0 input_3 vdd vdd sky130_fd_pr__pfet_01v8 w=22 l=2
X7 output_2 input_3 vdd gnd sky130_fd_pr__nfet_01v8 w=20 l=2
X8 output_1 internal_0 vdd vdd sky130_fd_pr__pfet_01v8 w=20 l=2

```

Fourth best validation reward circuit with train / validation / test reward of 0.747 / 0.747 / 0.702

```

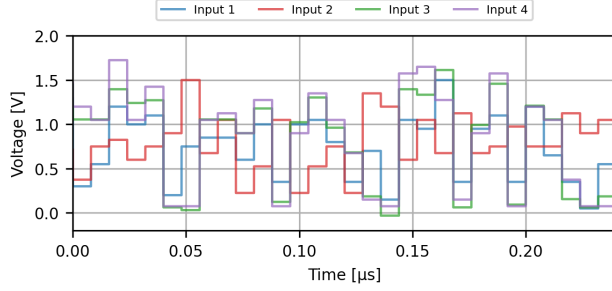
X0 output_1 input_1 gnd gnd sky130_fd_pr__nfet_01v8 w=10 l=1
X1 output_0 input_3 gnd gnd sky130_fd_pr__nfet_01v8 w=80 l=1
X2 output_1 input_3 gnd gnd sky130_fd_pr__nfet_01v8 w=2.17 l=0.42
X3 output_0 input_2 vdd vdd sky130_fd_pr__pfet_01v8 w=3 l=1
X4 output_1 input_1 vdd vdd sky130_fd_pr__pfet_01v8 w=3 l=1
X5 output_2 input_3 output_2 gnd sky130_fd_pr__nfet_01v8 w=1.5 l=1
X6 internal_0 internal_0 output_2 gnd sky130_fd_pr__nfet_01v8 w=1.3 l=9.35
X7 output_1 internal_0 vdd vdd sky130_fd_pr__pfet_01v8 w=10.9 l=46.5

```

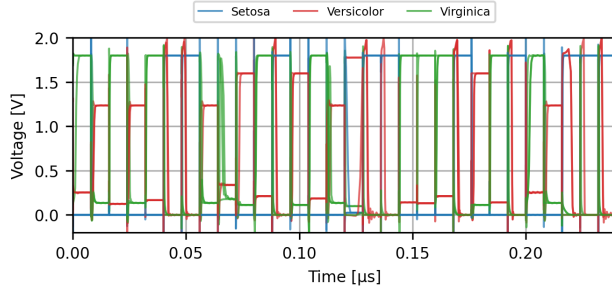
Figure 7: Netlists of additional circuits with strong validation performance.

C Output Waveforms Across Shuffles and Corners

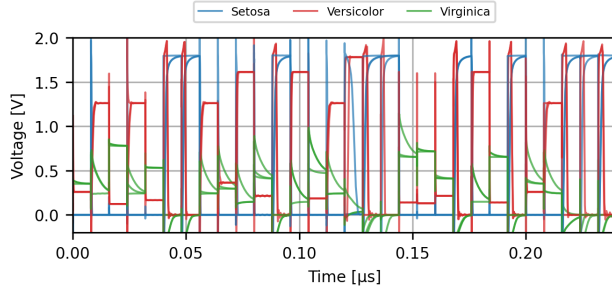
Fig. 9 shows the input and output waveforms for the two circuits with the best validation performance. The figure includes results for all considered corners and for all three shuffled versions of the test split. This provides additional insight into how the circuit behavior varies across operating conditions.



(a) Input waveforms

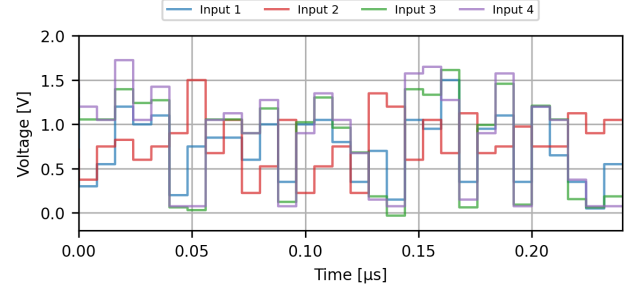


(b) Output waveforms (best validation circuit)

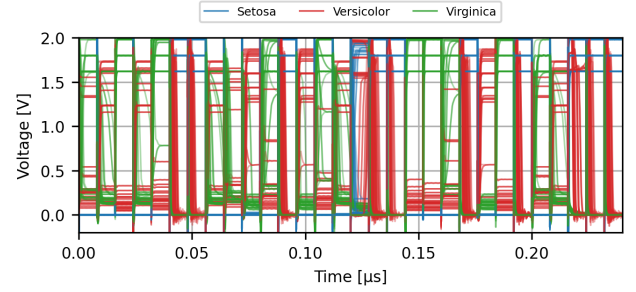


(c) Output waveforms (second-best validation circuit)

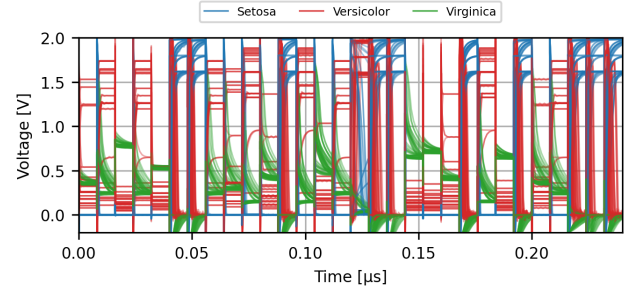
Figure 8: Voltage transients evaluated on the test split. The plots overlay the waveforms obtained from the three shuffled versions of the test split.



(a) Input waveforms



(b) Output waveforms (best validation circuit)



(c) Output waveforms (second-best validation circuit)

Figure 9: Voltage transients evaluated on the test split. The plots overlay the waveforms obtained for all corners and all three shuffled versions of the test split.

D Robustness to Input-Voltage Perturbations

Fig. 10 and Tab. 3 provide the detailed robustness results for the ML baselines and for the two best validation circuits found by LLM-SPICEMixer. We evaluate the classifiers both without input noise and under Gaussian perturbations of the input voltages, with $\sigma_{\text{noise}} \in \{0, 0.1, 0.2, 0.3, 0.4, 0.5\}$. For each noisy setting, results are aggregated over 16 independent noise realizations.

The boxplots in Fig. 10 show that the synthesized circuits are competitive with the ML baselines in the noiseless setting and exhibit a similar degradation trend as the input perturbation increases. Tab. 3 reports the corresponding numerical test accuracies, including mean, standard deviation, and median values. For LLM-SPICEMixer, we report both the nominal tt-corner performance and the average performance across all process, voltage, and temperature corners.

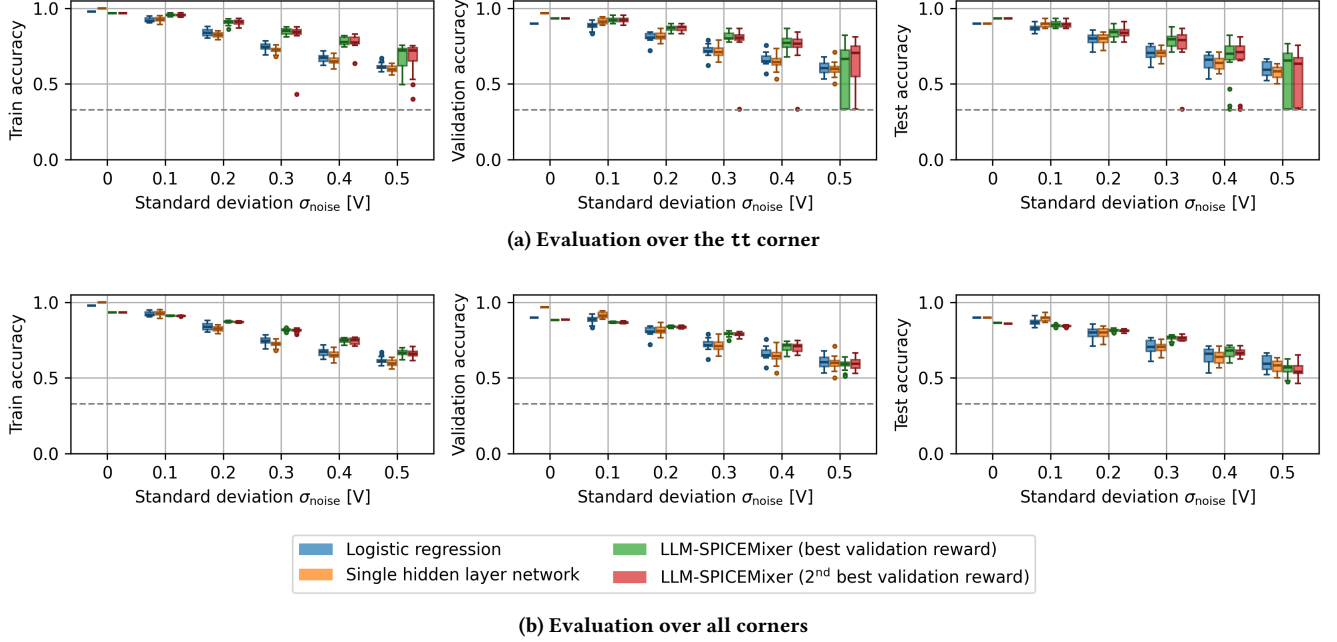


Figure 10: Robustness of LLM-SPICEMixer compared with ML baselines under input-voltage perturbations. The synthesized circuits are competitive with the baselines in the noiseless setting and degrade similarly as Gaussian noise σ_{noise} is added to the inputs, indicating that the analog solutions retain useful classification margins under imperfect sensor readings. Dashed lines represent chance accuracy.

Table 3: Test-set accuracy across noise levels. For LLM-SPICEMixer, both tt-only and all-corner results are shown. Each cell reports mean $\pm$ standard deviation / median, in percent (%).

| Method | Corners | $\sigma_{\text{noise}} = 0$ | $\sigma_{\text{noise}} = 0.1$ | $\sigma_{\text{noise}} = 0.2$ | $\sigma_{\text{noise}} = 0.3$ | $\sigma_{\text{noise}} = 0.4$ | $\sigma_{\text{noise}} = 0.5$ |
|--|---------|-----------------------------|-------------------------------|-------------------------------|-------------------------------|-------------------------------|-------------------------------|
| Logistic regression | N/A | 90.0 $\pm$ 0.0 / 90.0 | 87.0 $\pm$ 2.1 / 86.7 | 79.5 $\pm$ 3.7 / 80.0 | 70.8 $\pm$ 4.6 / 70.6 | 64.3 $\pm$ 5.2 / 66.1 | 59.8 $\pm$ 5.1 / 59.4 |
| Single-hidden-layer network | N/A | 90.0 $\pm$ 0.0 / 90.0 | 89.6 $\pm$ 1.9 / 90.0 | 79.7 $\pm$ 3.2 / 80.0 | 69.8 $\pm$ 3.5 / 70.6 | 63.7 $\pm$ 4.2 / 63.9 | 57.8 $\pm$ 4.0 / 58.3 |
| LLM-SPICEMixer (best validation) | tt | 93.3 $\pm$ 0.0 / 93.3 | 89.2 $\pm$ 1.9 / 89.4 | 84.1 $\pm$ 3.7 / 83.9 | 76.0 $\pm$ 12.0 / 78.9 | 65.6 $\pm$ 15.7 / 71.1 | 55.0 $\pm$ 15.9 / 63.3 |
| | all | 85.9 $\pm$ 0.0 / 85.9 | 84.2 $\pm$ 1.3 / 84.2 | 81.0 $\pm$ 2.6 / 81.1 | 74.8 $\pm$ 3.9 / 74.4 | 63.4 $\pm$ 8.5 / 63.4 | 52.1 $\pm$ 9.6 / 53.9 |
| LLM-SPICEMixer (2 nd best validation) | tt | 93.3 $\pm$ 0.0 / 93.3 | 90.0 $\pm$ 2.5 / 90.0 | 83.5 $\pm$ 4.1 / 85.0 | 78.2 $\pm$ 4.5 / 77.8 | 72.9 $\pm$ 4.3 / 73.3 | 60.3 $\pm$ 13.9 / 64.4 |
| | all | 87.5 $\pm$ 0.0 / 87.5 | 86.1 $\pm$ 1.8 / 86.3 | 82.0 $\pm$ 2.8 / 82.4 | 76.0 $\pm$ 4.5 / 76.5 | 67.5 $\pm$ 7.0 / 69.7 | 57.0 $\pm$ 9.0 / 59.4 |

E Results of Ablation Studies

Tabs. 4–7 provide the detailed results of the ablation studies for prompting strategy, decoding settings, model choice, and the role of the operator mixture. For each setting, the tables report the final best training reward over nine runs in terms of average, standard deviation, minimum, median, and maximum. In each ablation, we varied only one factor while keeping all other settings identical to the default IGEL configuration. For Tab. 4, Tab. 5, and Tab. 7, we used Gemma3 12B because of GPU resource constraints.

Tab. 7 compares IGEL-only search with the full operator mixture. To separate proposal budget from LLM-call budget, we report the full operator mixture both at the reduced proposal budget of 18,816 steps and at the full budget of 131,072 steps. The IGEL-only run uses 18,816 proposal steps, corresponding approximately to the number of LLM calls made in one full LLM-SPICEMixer run.

Table 4: Ablation: final best reward for different prompting strategies on the training split. Values are computed over nine independent runs.

| | without reasoning | | | with reasoning | | |
|---|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|
| | “raw”-style | “diff”-style | alternating | “raw”-style | “diff”-style | alternating |
| Average $\pm$ Std. Dev. | 0.731 $\pm$ 0.031 | 0.736 $\pm$ 0.020 | 0.733 $\pm$ 0.036 | 0.731 $\pm$ 0.038 | 0.745 $\pm$ 0.045 | 0.745 $\pm$ 0.042 |
| Minimum | 0.697 | 0.704 | 0.681 | 0.672 | 0.658 | 0.677 |
| Median | 0.713 | 0.745 | 0.729 | 0.750 | 0.754 | 0.763 |
| Maximum | 0.788 | 0.757 | 0.779 | 0.800 | 0.802 | 0.808 |

Table 5: Ablation: final best reward for different decoding settings on the training split. Values are computed over nine independent runs.

| | Deterministic | Conservative | Balanced | Entropy | Entropy++ |
|---|-------------------------------|---------------------------------|---------------------------------|----------------------------------|----------------------------------|
| | $T = 0, p_{\text{top}} = 1.0$ | $T = 0.3, p_{\text{top}} = 0.9$ | $T = 0.7, p_{\text{top}} = 0.9$ | $T = 1.0, p_{\text{top}} = 0.95$ | $T = 1.3, p_{\text{top}} = 0.98$ |
| Average $\pm$ Std. Dev. | 0.740 $\pm$ 0.028 | 0.734 $\pm$ 0.037 | 0.745 $\pm$ 0.042 | 0.726 $\pm$ 0.040 | 0.744 $\pm$ 0.040 |
| Minimum | 0.699 | 0.650 | 0.677 | 0.658 | 0.679 |
| Median | 0.741 | 0.756 | 0.763 | 0.748 | 0.755 |
| Maximum | 0.798 | 0.772 | 0.808 | 0.771 | 0.798 |

Table 6: Ablation: final best reward for different models on the training split. Values are computed over nine independent runs.

| | Gemma3 270M | Gemma3 1B | Gemma3 4B | Gemma3 12B | Gemma3 27B | Qwen3.5 9B | Qwen3.5 27B |
|---|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|
| Average $\pm$ Std. Dev. | 0.737 $\pm$ 0.034 | 0.740 $\pm$ 0.039 | 0.746 $\pm$ 0.028 | 0.745 $\pm$ 0.042 | 0.750 $\pm$ 0.019 | 0.743 $\pm$ 0.040 | 0.799 $\pm$ 0.040 |
| Minimum | 0.686 | 0.671 | 0.701 | 0.677 | 0.709 | 0.671 | 0.719 |
| Median | 0.737 | 0.749 | 0.744 | 0.763 | 0.760 | 0.750 | 0.810 |
| Maximum | 0.801 | 0.790 | 0.793 | 0.808 | 0.777 | 0.812 | 0.855 |

Table 7: Ablation: final best reward for Gemma3 12B when using the full operator mixture (= LLM-SPICEMixer) versus using only IGEL. The first two columns compare both settings at the same number of proposal steps. The last column reports the full all-operator run for reference; the IGEL-only setting uses approximately the same number of LLM calls as this full run.

| | All operators
18,816 steps
(16,128 SPICEMixer + 2,688 IGEL) | Only IGEL
18,816 steps
(0 SPICEMixer + 18,816 IGEL) | All operators
131,072 steps
(112,347 SPICEMixer + 18,725 IGEL) |
|---|---|---|--|
| | | | |
| Average $\pm$ Std. Dev. | 0.705 $\pm$ 0.031 | 0.607 $\pm$ 0.035 | 0.745 $\pm$ 0.042 |
| Minimum | 0.665 | 0.538 | 0.677 |
| Median | 0.689 | 0.619 | 0.763 |
| Maximum | 0.744 | 0.656 | 0.808 |

F LLM Response Length Statistics

Tab. 8 summarizes the response lengths of the evaluated LLMs in terms of both characters and words. For the Qwen3.5 models, we report thinking tokens and final output separately. These results provide additional context for the model comparison discussed in the main paper.

Table 8: Response-length statistics across the four models. For the Qwen3.5 models, values are reported as <think> + final output, because these models produce explicit thinking tokens.

| | Gemma3 12B | Gemma3 27B | Qwen3.5 9B | Qwen3.5 27B |
|-----------------------------|-----------------|-----------------|--|---|
| <i>Number of characters</i> | | | | |
| Average ± Std. Dev. | 1,576.7 ± 389.2 | 1,215.9 ± 336.0 | 5,401.5 ± 4,217.8
+ 1,029.4 ± 625.6 | 21,825.9 ± 8,690.7
+ 1,322.9 ± 725.3 |
| Minimum | 430 | 263 | 1,173 + 61 | 1,216 + 62 |
| Median | 1,532 | 1,176 | 3,961 + 894 | 23,871 + 1,336 |
| Maximum | 3,780 | 16,977 | 59,653 + 23,750 | 69,535 + 23,088 |
| <i>Number of words</i> | | | | |
| Average ± Std. Dev. | 226.2 ± 54.1 | 177.1 ± 46.8 | 793.1 ± 627.2
+ 141.1 ± 89.8 | 3,368.5 ± 1,364.4
+ 186.4 ± 102.9 |
| Minimum | 65 | 42 | 137 + 10 | 170 + 10 |
| Median | 221 | 172 | 576 + 121 | 3,675 + 189 |
| Maximum | 498 | 2,333 | 9,121 + 3,094 | 7,205 + 3,170 |